

MATERIAL EXTRA

Trabalhando com Funções no Typescript



INTRODUÇÃO

Se você sabe escrever funções em JavaScript, também saberá escrever funções no TypeScript. Mas o TypeScript adiciona alguns recursos novos às funções JavaScript padrão para facilitar o trabalho com elas.

No JavaScript, as definições de função não especificam os tipos de dados para parâmetros, não executam a verificação de tipo nos argumentos transmitidos nem verificam a quantidade de argumentos recebidos. Portanto, você precisa adicionar a lógica para verificar esses parâmetros em suas funções.

O TypeScript simplifica o desenvolvimento de funções e as torna mais fáceis de solucionar, permitindo que você digite parâmetros e retorne valores. O TypeScript também adiciona novas opções para parâmetros. Por exemplo, embora todos os parâmetros sejam opcionais em funções JavaScript, você pode torná-los obrigatórios ou opcionais no TypeScript.

Adicionar tipos às funções ajuda a evitar que você passe valores que não deveria para as suas funções. Funções tipadas são especialmente importantes quando você está trabalhando com bases de código maiores ou funções desenvolvidas por outras pessoas. Por mais que a adição de tipos seja uma diferença simples, ela oferece a vantagem de verificar o tipo dos valores transmitidos para a função e o que é retornado. Em vez de ter que ficar adicionando toda a lógica à função para verificar se o tipo de valor correto foi transmitido a ela e se o valor de retorno está correto, o TypeScript ajuda a garantir os tipos de valor corretos conforme você desenvolve seu código. Além disso, ao criar a lógica de função, você terá suporte completo para preenchimento automático, pois o editor conhecerá o tipo de dados dos parâmetros – algo que o JavaScript

normalmente não consegue detectar. O preenchimento automático é especialmente útil quando você está usando funções desenvolvidas por outras pessoas porque o TypeScript esclarece os tipos de entrada e saída necessários.

Assim como no JavaScript, você pode definir funções no TypeScript de várias maneiras diferentes. Vejamos como essas funções se diferem com a adição de tipos no TypeScript.

FUNÇÕES NOMEADAS

Uma função nomeada é uma declaração de função escrita com a palavra-chave `function` e com um nome distinto dentro do escopo atual. As declarações de funções nomeadas são carregadas no contexto de execução antes da execução de qualquer código. Esse processo é conhecido como guindaste e significa que você pode usar a função antes de declará-la.

A sintaxe para declarar uma função nomeada no TypeScript é a mesma usada para definir uma no JavaScript. A única diferença para o TypeScript é que você pode fornecer uma anotação de tipo para os parâmetros e o valor retornado da função.

Essa função aceita dois parâmetros do tipo `number` e retorna um `number`.

```
function addNumbers(x: number, y: number): number {  
    return x + y;  
}  
  
addNumbers(1, 2);
```

FUNÇÕES ANÔNIMAS

Uma **expressão de função** (ou **função anônima**) é uma função que não é pré-carregada no contexto de execução e só é executada quando o código a encontra. As expressões de função são criadas em runtime e devem ser declaradas antes que possam ser chamadas. Elas não são içadas, ao contrário das declarações de funções nomeadas que são içadas assim que a execução do programa começa e podem ser chamadas antes de sua declaração.

As expressões de função representam valores; portanto, são geralmente atribuídas a uma variável ou transmitidas para outras funções e podem ser anônimas, o que significa que a função não tem nome.

Este exemplo atribui uma expressão `function` à variável `addNumbers`. Observe que a função aparece no lugar do nome da função, tornando a função anônima. Você já pode usar essa variável para chamar a função.

```
let addNumbers = function (x: number, y: number): number {  
    return x + y;  
};  
addNumbers(1, 2);
```

O exemplo a seguir mostra a aparência da função nomeada `sum` quando escrita como uma função anônima. O nome `sum` foi substituído pela palavra-chave `function`, e a função foi implementada como uma expressão em uma declaração de variável.

```
let sum = function (input: number[]): number {  
  let total: number = 0;  
  for (let i = 0; i < input.length; i++) {  
    if (isNaN(input[i])) {  
      continue;  
    }  
    total += Number(input[i]);  
  }  
  return total;  
};  
  
console.log(sum([1, 2, 3]));
```

Assim como antes, você obterá a verificação de tipo e o IntelliSense ao usar funções anônimas. Você também observará, nesse exemplo, que a variável `sum` não é tipada, mas o TypeScript é capaz de determinar seu tipo através de algo chamado "tipificação contextual", uma forma de inferência de tipos. A tipagem contextual pode reduzir a quantidade de esforço necessária para manter seu programa tipado.

FUNÇÕES DE SETA

As funções de seta (também chamadas de funções Lambda ou de seta grossa, devido ao operador `=>` usado para defini-las) fornecem uma sintaxe abreviada para definir uma função anônima. Devido à sua natureza concisa, as funções de seta geralmente são usadas com funções simples e em alguns cenários de manipulação de eventos.

Este exemplo compara a sintaxe de uma `function` anônima com uma função de seta de linha única. A função de seta abrevia a sintaxe omitindo a palavra-chave da função e adicionando o operador `⇒` entre os parâmetros e o corpo da função.

```
// Anonymous function
let addNumbers1 = function (x: number, y: number): number {
  return x + y;
}

// Arrow function
let addNumbers2 = (x: number, y: number): number ⇒ x + y;
```

Nesse exemplo, observe ainda que as chaves foram removidas e que não há uma instrução `return`. As funções de seta de linha única podem usar uma **sintaxe de corpo concisa** ou um **retorno implícito**, que permite a omissão das chaves e da palavra-chave `return`.

Se o corpo da função tiver mais do que uma única linha, use chaves em torno dela e inclua a instrução `return` (se for o caso). Este exemplo mostra a aparência da função anônima do exemplo anterior quando gravada como função de seta.

```
let total2 = (input: number[]): number => {  
  let total: number = 0;  
  for (let i = 0; i < input.length; i++) {  
    if (isNaN(input[i])) {  
      continue;  
    }  
    total += Number(input[i]);  
  }  
  return total;  
};
```

TRABALHAR COM PARÂMETROS

O compilador TypeScript pressupõe, por padrão, que todos os parâmetros definidos em uma função são obrigatórios. Quando uma função é chamada, o compilador TypeScript verifica se:

- Cada parâmetro recebeu um valor.
- Somente os parâmetros que a função requer são transmitidos para ela.
- Os parâmetros são passados na ordem em que são definidos na função.

Esses requisitos são diferentes do JavaScript. O JavaScript assume que todos os parâmetros são opcionais e permite que você passe mais (ou menos) argumentos para a função do que os definidos por ela.

Além dos parâmetros obrigatórios, você pode definir funções com parâmetros opcionais, padrão e rest, bem como parâmetros de objeto desconstruídos.

Parâmetros obrigatórios

Todos os parâmetros de função são obrigatórios, salvo se especificado de outra forma, e a quantidade de argumentos transmitidos a uma função deve corresponder ao número de parâmetros obrigatórios que a função espera. Neste exemplo, todos os parâmetros são obrigatórios.

```
function addNumbers (x: number, y: number): number {  
    return x + y;  
}  
  
addNumbers(1, 2); // Returns 3  
addNumbers(1);   // Returns an error
```

Parâmetros opcionais

Também é possível definir parâmetros opcionais adicionando um ponto de interrogação (?) ao final do nome do parâmetro.

Neste exemplo, `x` é obrigatório e `y` é opcional. O parâmetro opcional precisa vir depois dos parâmetros obrigatórios na lista de parâmetros. Além disso, para que essa função retorne o valor correto, você também precisa abordar a possibilidade de que `y` já tenha sido transmitido como indefinido.

```
function addNumbers (x: number, y?: number): number {  
  if (y === undefined) {  
    return x;  
  } else {  
    return x + y;  
  }  
}  
  
addNumbers(1, 2); // Returns 3  
addNumbers(1);    // Returns 1
```

Parâmetros padrão

Também é possível atribuir valores padrão a parâmetros opcionais. Se um valor for transmitido como argumento para o parâmetro opcional, esse valor será atribuído a ele. Caso contrário, o valor padrão será atribuído. Assim como ocorre com parâmetros opcionais, os parâmetros padrão devem vir após os obrigatórios na lista de parâmetros.

Neste exemplo, *x* é obrigatório e *y* é opcional. Se um valor não for passado para *y*, o valor padrão será 25.

```
function addNumbers (x: number, y = 25): number {  
  return x + y;  
}  
  
addNumbers(1, 2); // Returns 3  
addNumbers(1);    // Returns 26
```

Parâmetros rest

Se você quiser trabalhar com vários parâmetros como um grupo (por exemplo, passando-os em uma matriz). Ou, se você não souber quantos parâmetros uma função levará em última análise. Você pode usar parâmetros rest. Os parâmetros rest são tratados como um número ilimitado de parâmetros opcionais. Você pode deixá-los desativados ou usar o máximo que desejar.

Este exemplo tem um parâmetro obrigatório e um parâmetro opcional chamado `restOfNumbers` que pode aceitar qualquer quantidade adicional de números. As reticências (...) antes de `restOfNumbers` informam ao compilador para criar uma matriz dos argumentos transmitidos à função e atribui o nome que se segue a ela para que você possa usá-la em sua função.

```
function addAllNumbers(  
  firstNumber: number,  
  ...restOfNumbers: number[]  
) : number {  
  let total: number = firstNumber;  
  for (let counter = 0; counter < restOfNumbers.length; counter++) {  
    if (isNaN(restOfNumbers[counter])) {  
      continue;  
    }  
    total += Number(restOfNumbers[counter]);  
  }  
  return total;  
}
```

A função agora pode aceitar um ou mais valores e retornar o resultado.

```
addAllNumbers(1, 2, 3, 4, 5, 6, 7); // returns 28
addAllNumbers(2);                    // returns 2
addAllNumbers(2, 3, "three");        // flags error due to data type
at design time, returns 5
```

Parâmetros de objeto desconstruídos

Os parâmetros de função são posicionais e devem ser passados na ordem em que são definidos na função. Parâmetros posicionais podem tornar seu código menos legível ao chamar uma função com vários parâmetros opcionais ou do mesmo tipo de dados.

Para habilitar parâmetros nomeados, você pode usar uma técnica chamada parâmetros de objeto desconstruído. Essa técnica permite que você use uma interface para definir parâmetros nomeados, em vez de posicionais, em suas funções.

O exemplo a seguir define uma interface denominada `Message` que define duas propriedades. Na função `displayMessage`, o objeto `Message` é passado como parâmetro, dando acesso às propriedades como se fossem parâmetros normais.

```
interface Message {  
  text: string;  
  sender: string;  
}  
  
function displayMessage({ text, sender }: Message) {  
  console.log(`Message from ${sender}: ${text}`);  
}  
  
displayMessage({  
  sender: "Christopher",  
  text: "hello, world"  
});
```

BONS ESTUDOS